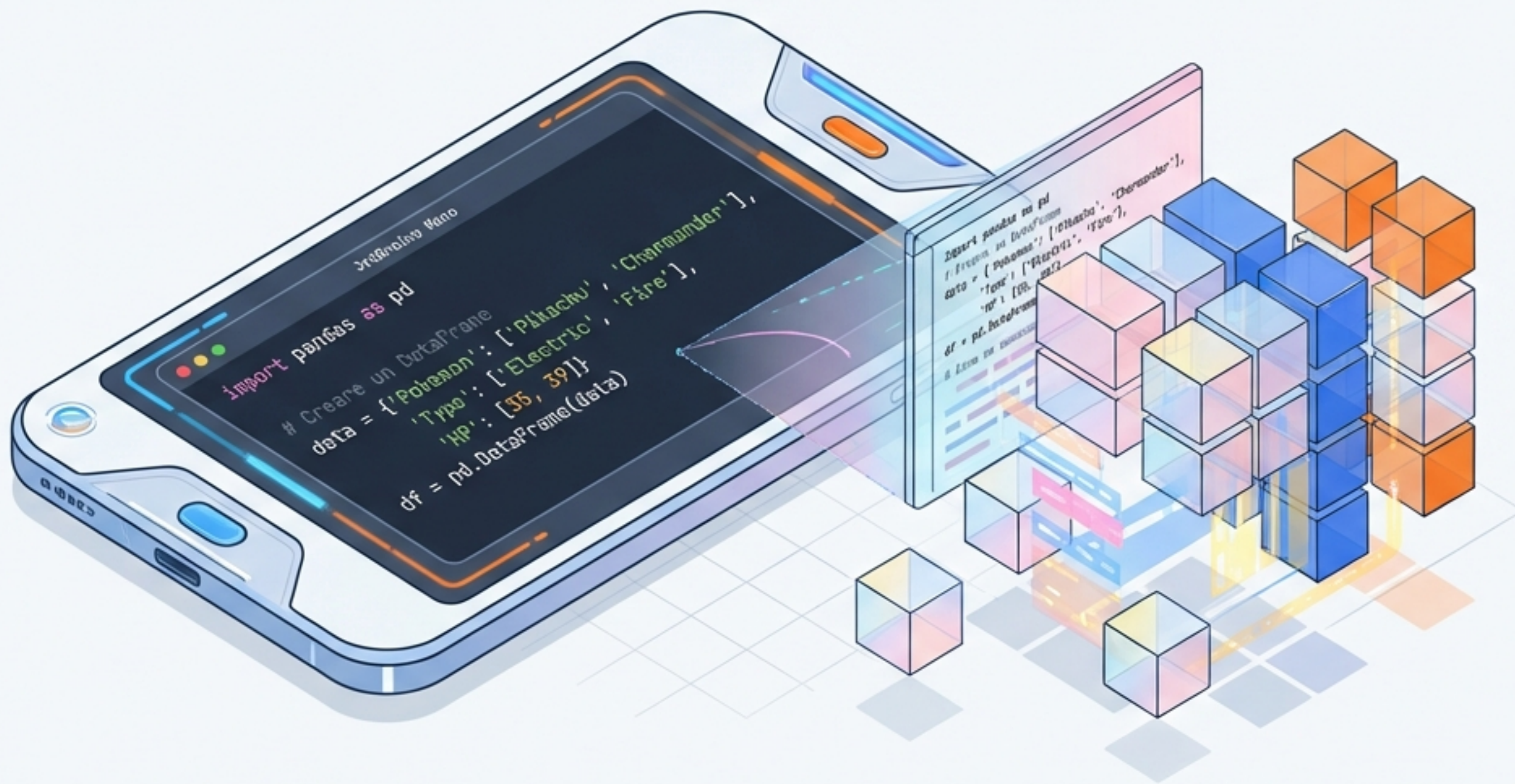


Padroneggiare Pandas: Il Manuale del Data Trainer

Dalle Serie ai DataFrame: Guida completa alla **manipolazione dei dati** in **Python**.



L'Equipaggiamento: Installazione e Importazione



The Terminal

```
# Nel terminale  
$ pip install pandas
```

```
# Nello script Python  
import pandas as pd
```

```
>_
```



The Explanation



Installazione

Se l'importazione fallisce, esegui il comando pip nel terminale o nel prompt dei comandi.



Alias Standard

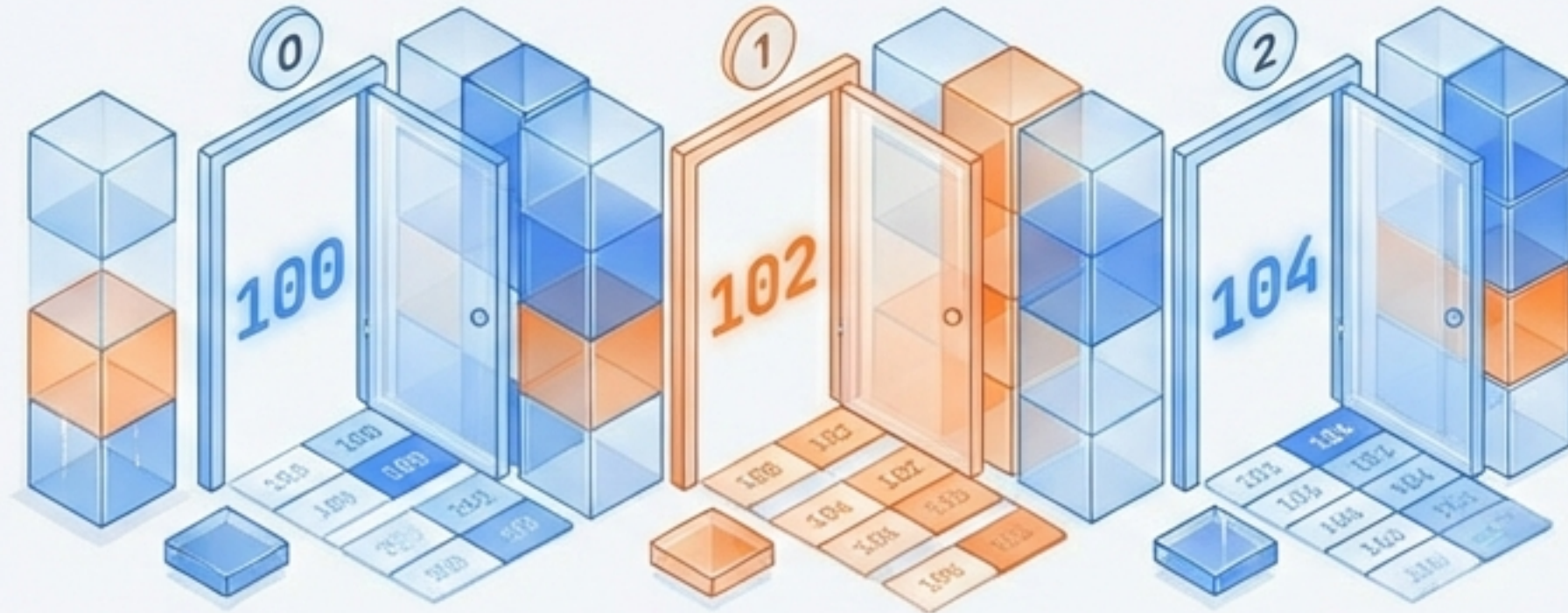
Usiamo pd come abbreviazione convenzionale. Rende il codice più pulito e veloce da scrivere.



Verifica Versione

Controlla l'installazione con: `print(pd.__version__)`

La Serie: Un Corridoio di Dati Unidimensionale



Codice

```
voti = [100, 102, 104]
series = pd.Series(voti)

print(series)
```



Output Console

```
0    100
1    102
2    104
dtype: int64
```

Una Series è una lista potenziata: possiede un **Indice** (sinistra) e i **Valori** (destra).

Personalizzare l'Indice

Gli indici non devono essere per forza numeri. Possiamo usare etichette significative.

```
# Assegnazione di etichette personalizzate (Index)
series = pd.Series(voti, index=["Alunno A", "Alunno B", "Alunno C"])

# Accesso ai dati tramite etichetta
print(series["Alunno A"])
```



Indice (Index)	Valore (Value)
Alunno A	100
Alunno B	102
Alunno C	104

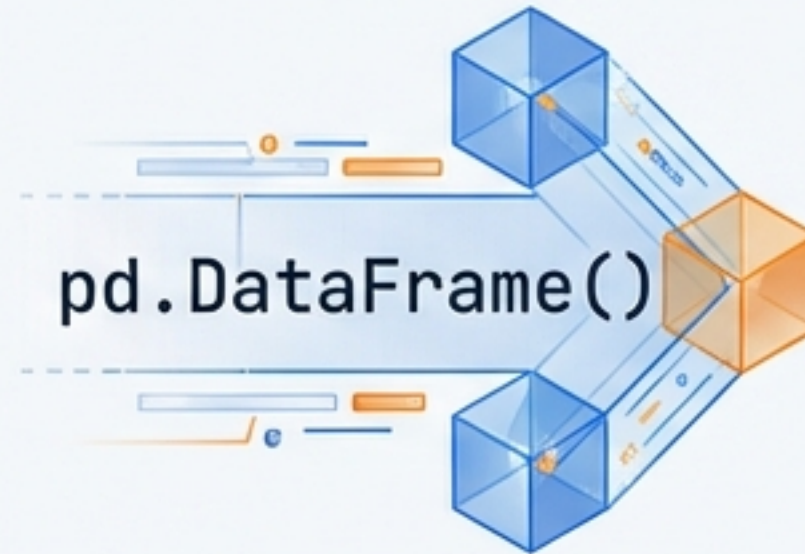


Accesso diretto al valore tramite etichetta!

Il DataFrame: La Struttura Tabellare

Un DataFrame è una griglia bidimensionale di righe e colonne.
Si crea facilmente da un Dizionario Python.

```
data = {  
    "Nome": ["Spongebob",  
            "Patrick", "Squidward"],  
    "Età": [30, 35, 50]  
}  
  
df = pd.DataFrame(data)
```



	Nome	**Età**
0	Spongebob	30
1	Patrick	35
2	Squidward	50

Le chiavi del dizionario diventano le colonne. Gli indici (0, 1, 2) sono generati automaticamente.

Evoluzione della Tabella: Aggiungere Dati

1. Aggiungere una Colonna

```
df["Lavoro"] = ["Cuoco", "Disoccupato", "Cassiere"]
```



	Nome	**Età**	**Lavoro**
0	Spongebob	30	Cuoco
1	Patrick	35	Disoccupato
2	Squidward	50	Cassiere

2. Aggiungere una Riga (Concatenazione)

```
nuova_riga = pd.DataFrame([{"Nome": "Sandy",  
                             "Età": 28, "Lavoro": "Scienziata"}])  
df = pd.concat([df, nuova_riga], ignore_index=True)
```



	Nome	**Età**	**Lavoro**
0	Spongebob	30	Cuoco
1	Patrick	35	Disoccupato
2	Squidward	50	Cassiere
3	Sandy	28	Scienziata

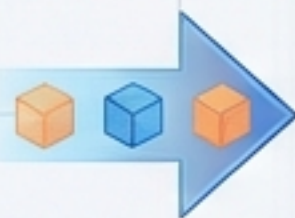
⚠ Nota: `append()` è deprecato. Usa `concat()`.

Acquisizione Dati: Leggere CSV e JSON

Importare il dataset dei 150 Pokémon Originali.

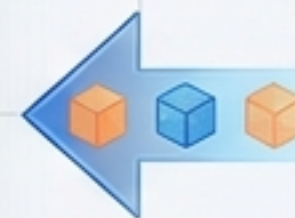


pokemon_data.csv



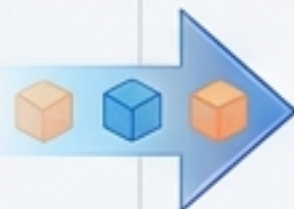
```
# Lettura CSV
df = pd.read_csv('pokemon_data.csv')

# Lettura JSON
df_json = pd.read_json('pokemon_data.json')
```



pokemon_data.json

Name, Type 1, Type 2, Height, Weight, Legendary



Name	Type 1	Type 2	Height	Weight	Legendary
------	--------	--------	--------	--------	-----------

Ispezione del Dataset

Pandas nasconde le righe centrali per risparmiare spazio. Usa `to_string()` per vedere tutto.

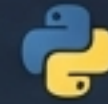
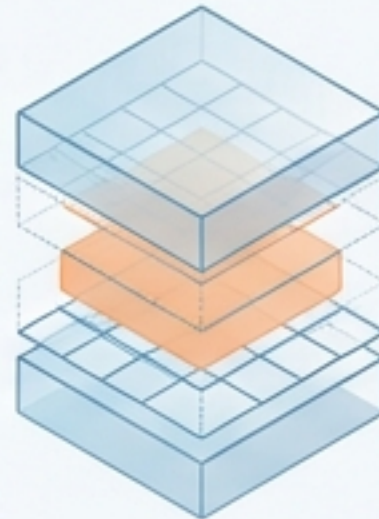


```
print(df)
```

Default
View

	Name	Type 1	...
0	Bulbasaur	Grass	...
1	Ivysaur	Grass	...
...	...	...	...
149	Mewtwo	Psychic	...

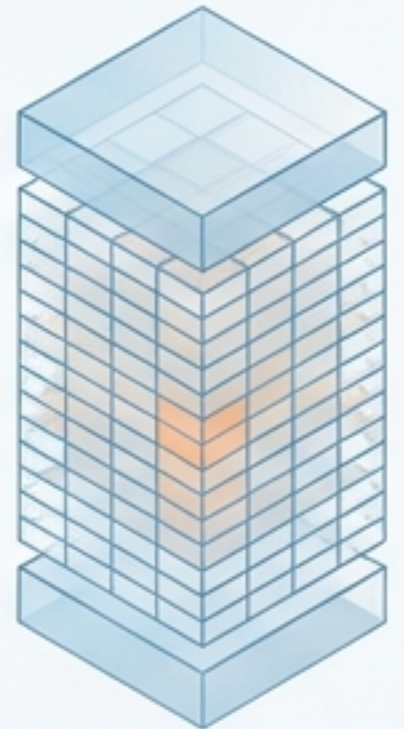
[150 rows x 6 columns]



```
print(df.to_string())
```

Full
View

0	Name	Type 1	...
0	Bulbasaur	Grass	...
1	Ivysaur	Grass	...
...	...	...	...
...	...	...	...
149	Mewtwo	Psychic	...
5	Hyperinan	Grass	...
6	Haspot	Psychic	...
7	Pabbit	Psychic	...
8	Pandat	Psychic	...
9	Montano	Grass	...
100	Marcin	Psychic	...
111	Holtesto	Psychic	...
112	Rausn	Psychic	...
113	Lordson	Psychic	...
144	Shopotap	Grass	...
145	Tarmon	Psychic	...
149	Mewtwo	Psychic	...

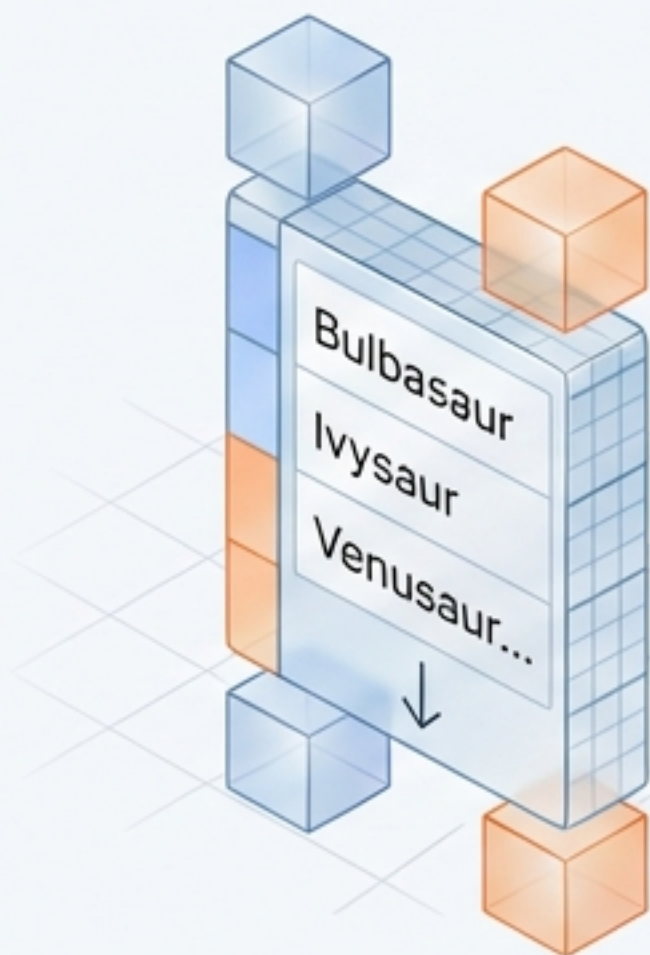


Attenzione con file molto grandi!

Selezione per Colonna

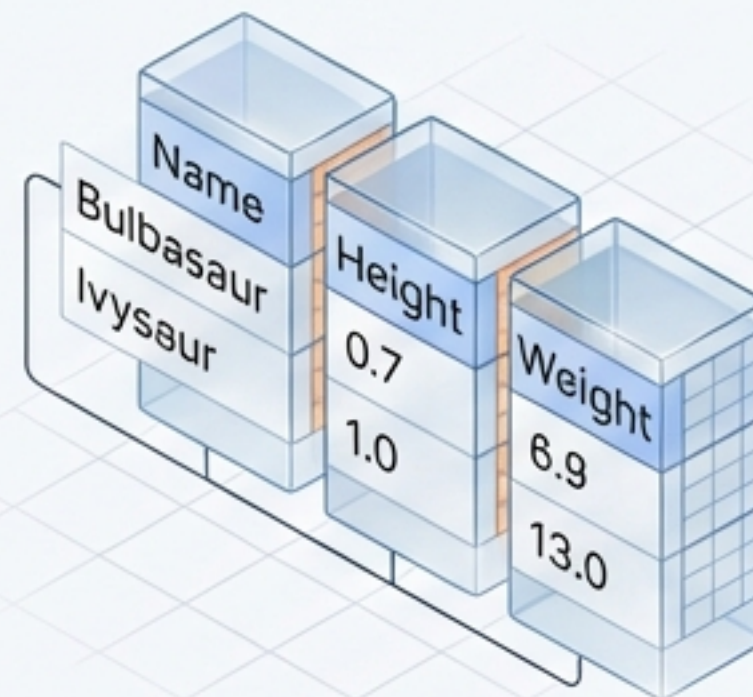
Singola Colonna (Series)

```
df["Name"]
```



Multiple Colonne (DataFrame)

```
df[["Name", "Height", "Weight"]]
```



Navigare le Righe: La Battaglia tra LOC e ILOC

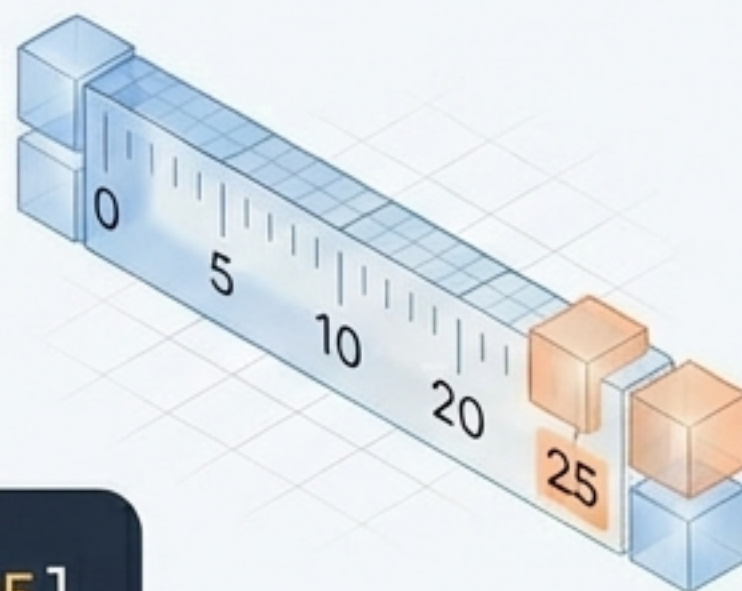
****1-6**loc** (Location by Label)



```
df.loc["Pikachu"]
```

Cerca tramite l'etichetta dell'indice (**Stringa**).
(Richiede ``index_col='Name'``).

****iloc** (Integer Location)



```
df.iloc[25]
```

Cerca tramite la posizione numerica assoluta (**0-N**).

```
Output: Name: Pikachu, Type: Electric, Height: 0.4
```

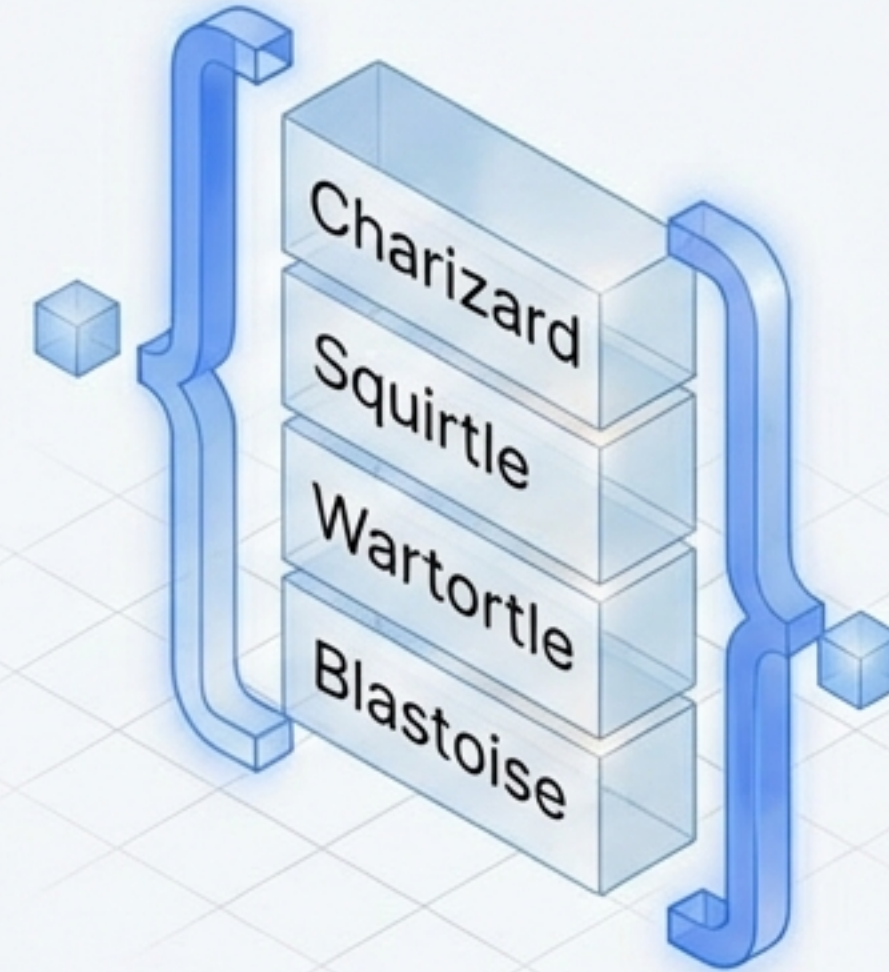

Slicing e Ricerca Sicura

Slicing (Intervalli)

```
df.loc["Charizard":"Blastoise"]
```

Gestione Errori (Input Utente)

```
try:  
    name = input("Inserisci Pokémon: ")  
    print(df.loc[name])  
except KeyError:  
    print("Pokémon non trovato!")
```



Il blocco **try/except** evita che il programma crashi se l'utente digita un nome inesistente.

Filtraggio Logico: Trovare i Campioni

Il filtraggio restituisce solo le righe che soddisfano una condizione (TRUE).

I Giganti (Altezza)

```
df.loc[df["Height"] >= 2.0]
```



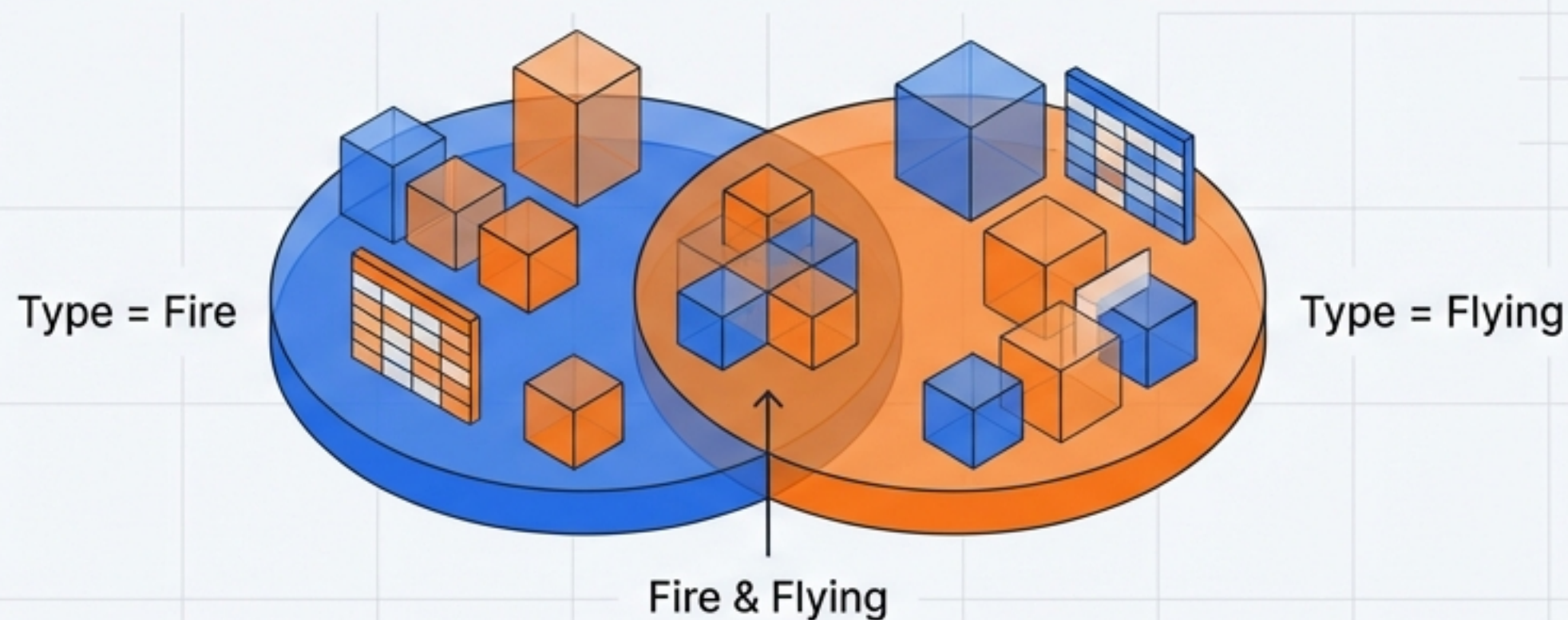
I Leggendarî

```
df.loc[df["Legendary"] == 1]
```



Logica Avanzata: AND (&) vs OR (|)

Attenzione: Usa gli operatori bitwise `&` e `|`, non le parole chiave Python `and` / `or`. Le parentesi sono obbligatorie.



1. **OR** (Unione):

```
df.loc[(df["Type 1"] == "Water") | (df["Type 2"] == "Water")]
```

—————> Unione of both

2. **AND** (Intersezione):

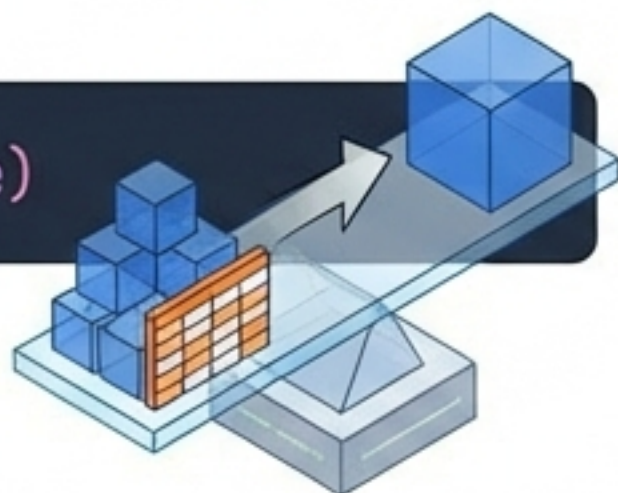
```
df.loc[(df["Type 1"] == "Fire") & (df["Type 2"] == "Flying")]
```

—————> Charizard, Moltres

Analisi Statistica: Aggregazione

Media (Mean)

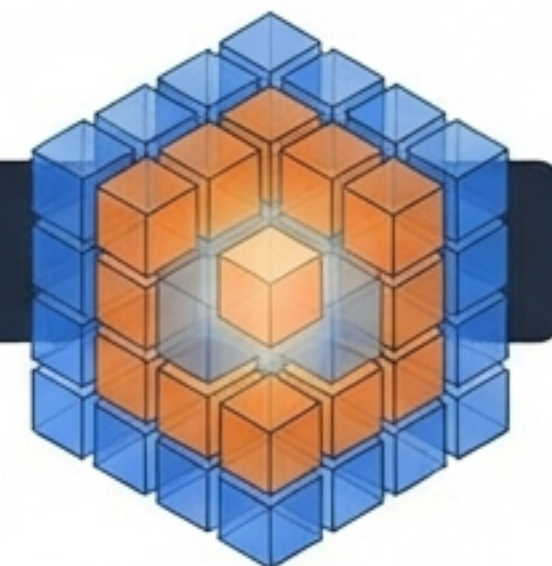
```
df.mean(numeric_only=True)
```



Height: 1.19m

Somma (Sum)

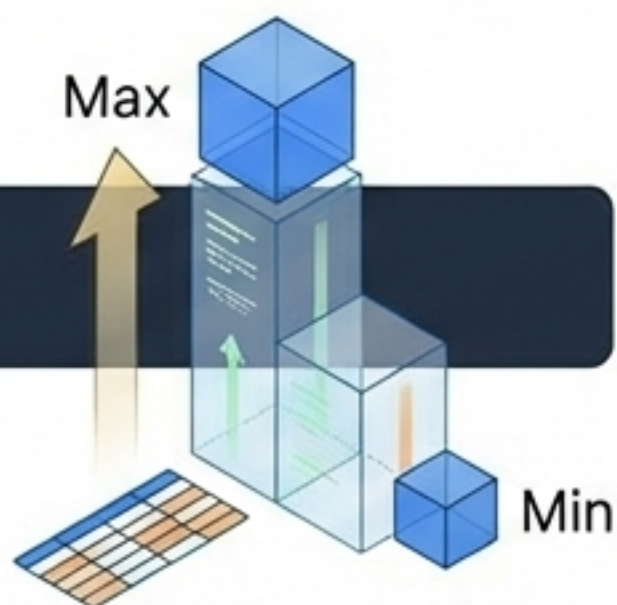
```
df["Weight"].sum()
```



Total Weight: 6938.0 kg

Estremi (Min/Max)

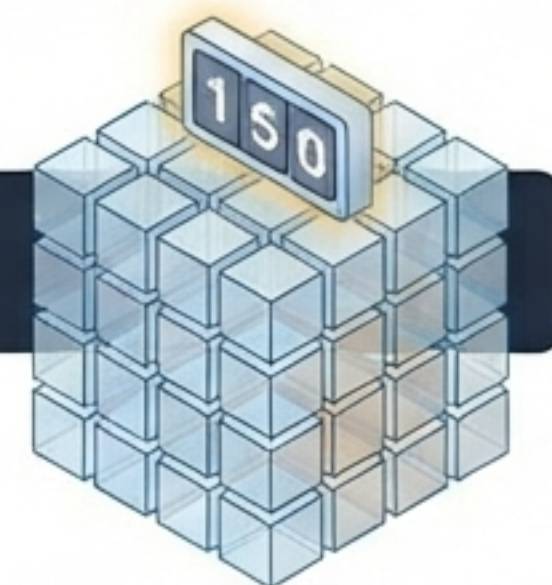
```
df["Height"].max()
```



8.8m (Onix)

Conteggio (Count)

```
df.count()
```



Non-null values: 150

GroupBy: Analisi per Categoria

Raggruppa le righe che hanno valori in comune (es. **Tipo**) per calcolare **statistiche di gruppo**.

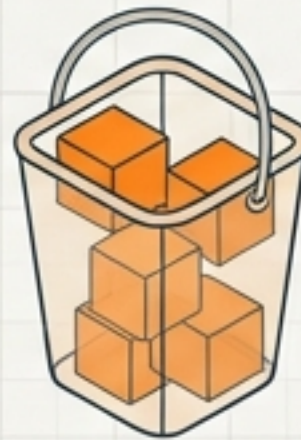


Raw Data
(Mixed Types)

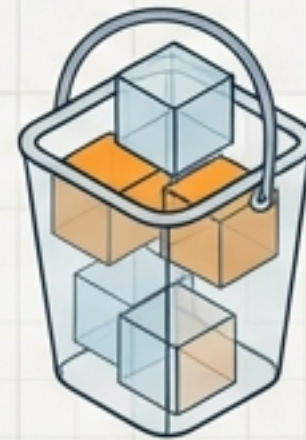
`.groupby("Type 1")`



Bug



Fire



Dragon

`.mean()`

Type 1	
Bug	0.92
Dragon	2.65
Electric	0.88
...	

I draghi sono i più
alti in media!

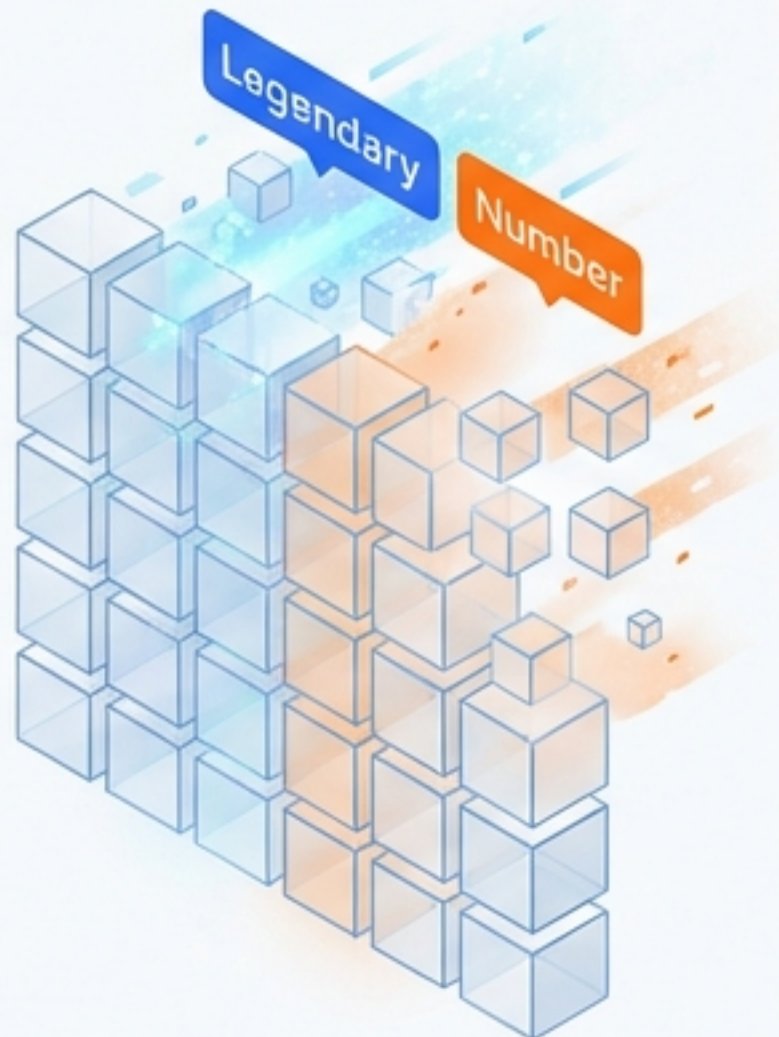
```
print(df.groupby("Type 1")["Height"].mean())
```


Pulizia Dati: Rimozione (Drop)

“Il 75% del lavoro è pulizia.”

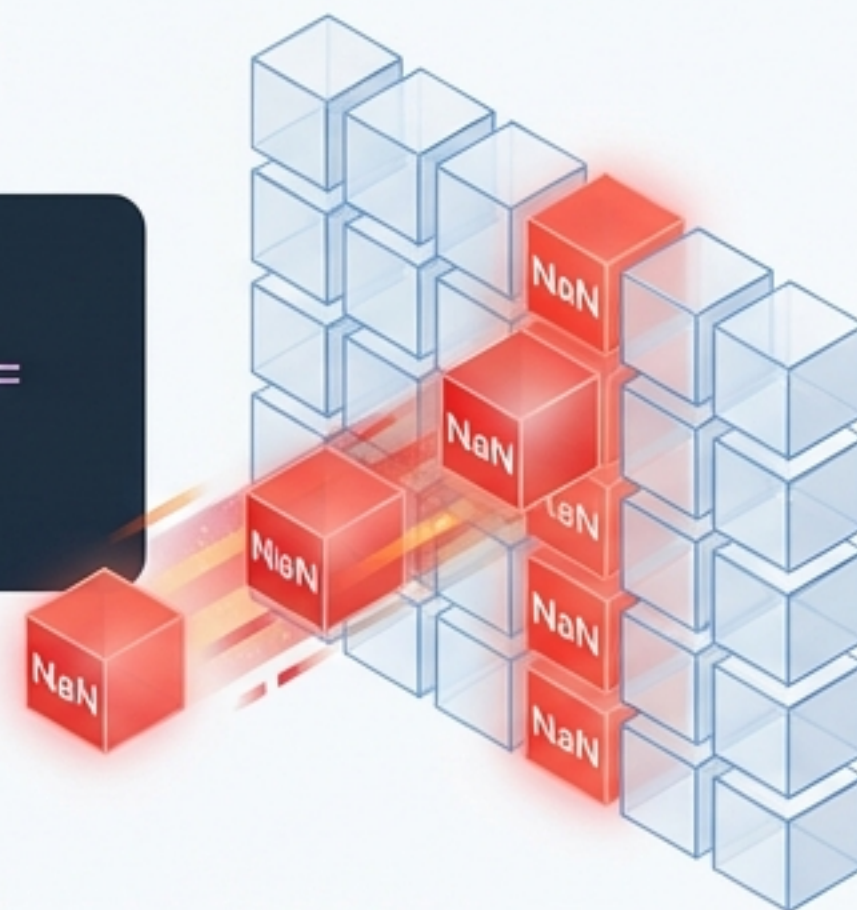
Cancellare Colonne

```
df.drop(columns=[  
    "Legendary",  
    "Number"])
```



Cancellare Righe (Nulls)

```
df.dropna(subset=[  
    "Type 2"])
```



Rimuove Pokémon come Charmander (che non ha un secondo tipo).

Pulizia Dati: Riempimento (FillNA)

Invece di **cancellare** i dati mancanti (NaN), **sostituiscili** con **un valore neutro**.

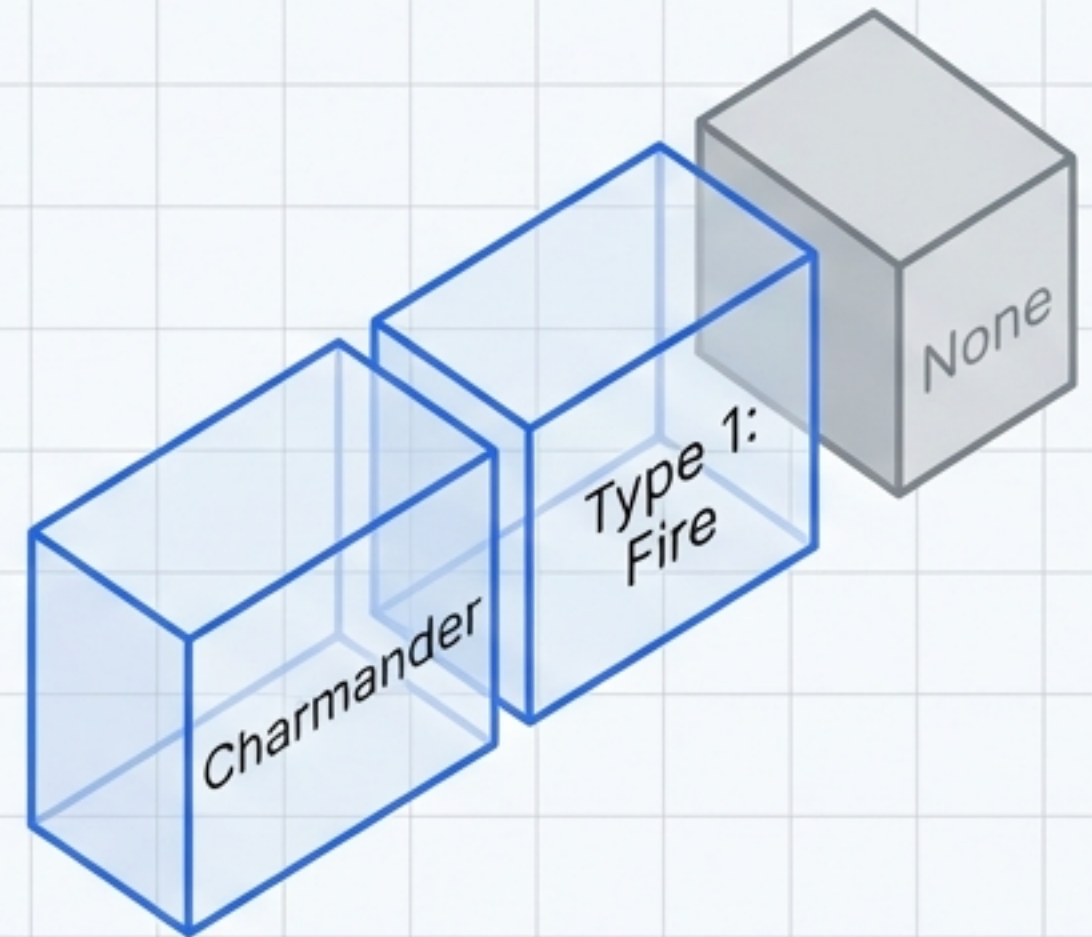


```
df["Type 2"] = df["Type 2"].fillna("None")
```

Prima



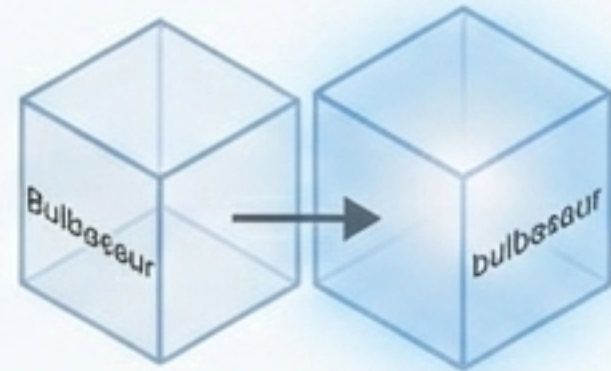
.fillna('None')



Standardizzazione e Correzione Tipi

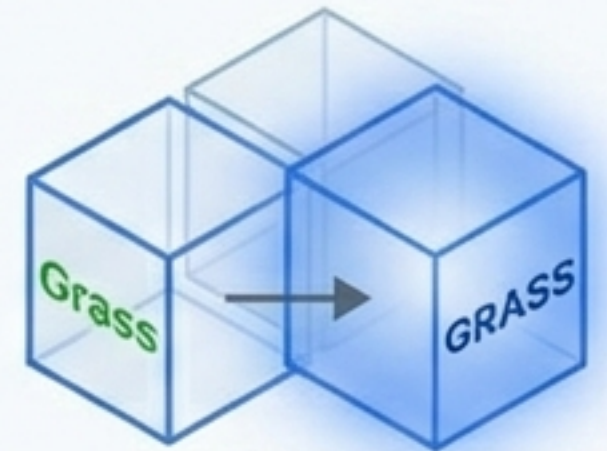
Testo (Lowercase)

```
df["Name"].str.lower()
```



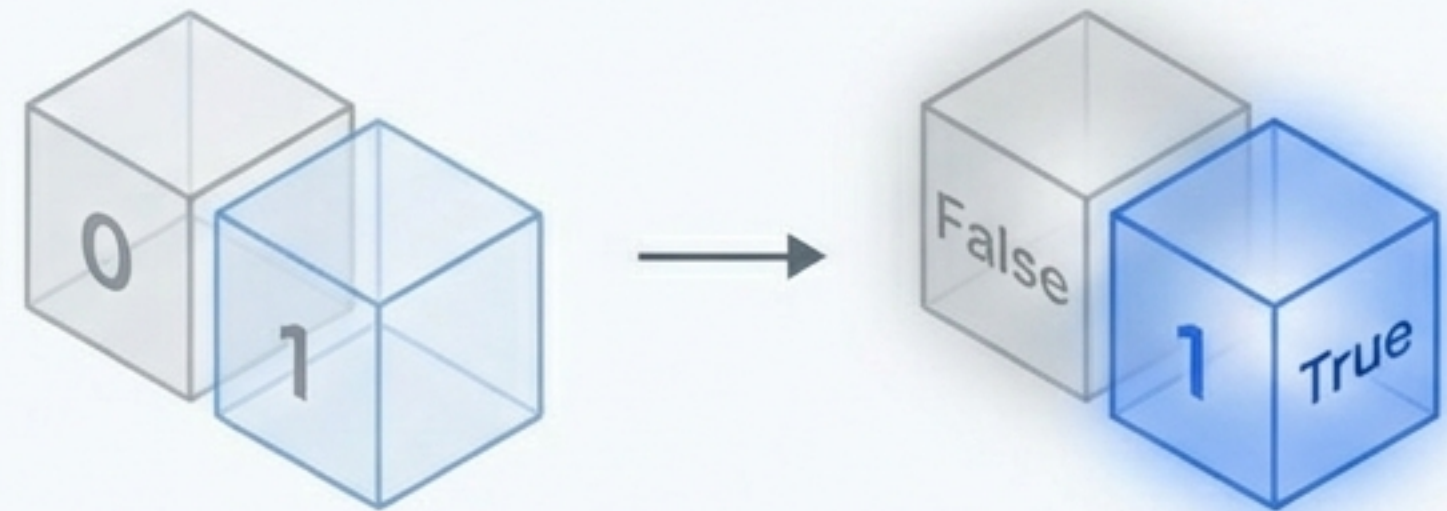
Sostituzione Valori

```
df["Type 1"].replace  
({"Grass": "GRASS"})
```



Conversione Tipi (Type Casting)

```
df["Legendary"].astype(bool)
```



Rimozione dei Duplicati

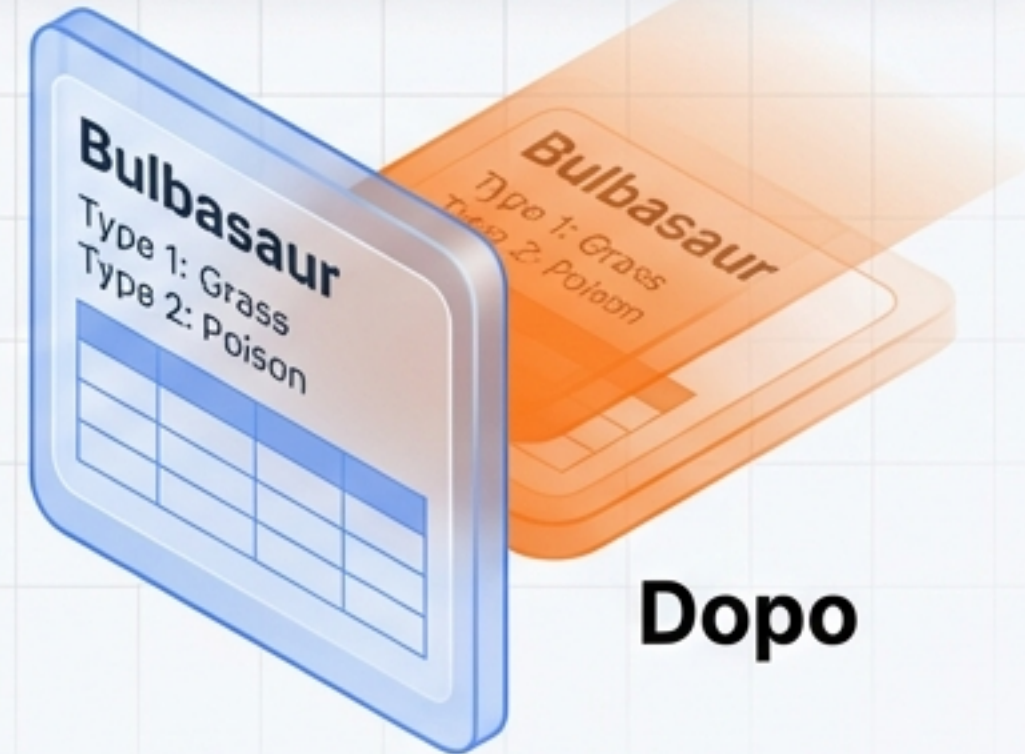
Errori di copia/incolla possono creare righe identiche.

Prima



```
df = df.drop_duplicates()
```

.drop_duplicates()



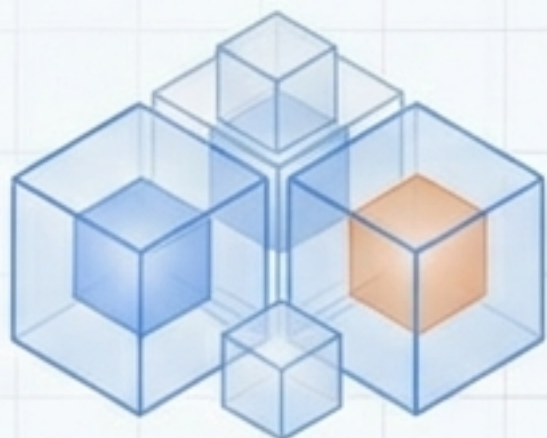
Dopo

Mantiene solo la **prima** occorrenza. Il dataset torna a **150 righe**.

Il Kit del Data Trainer (Riepilogo)

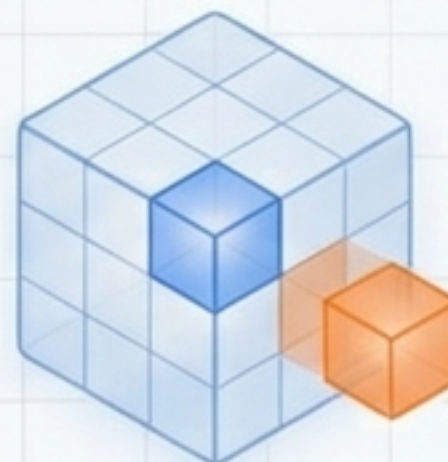
Setup

```
pd.read_csv()  
pd.DataFrame()
```



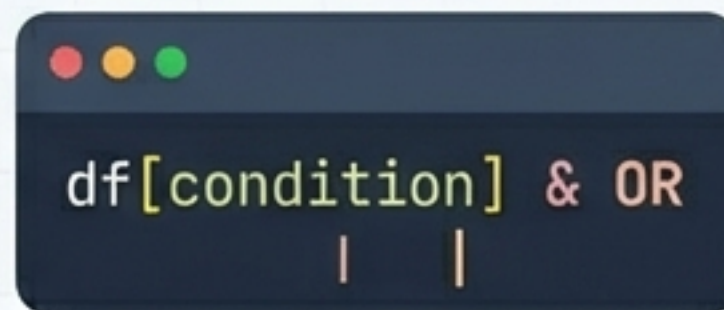
Selezione

```
df[col]  
df.loc[label]  
df.iloc[index]
```



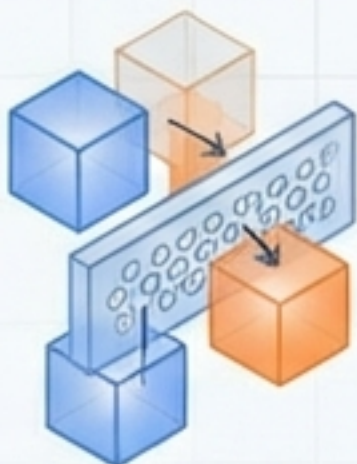
Geniat

```
df[condition] & OR  
| |
```



Filtri

```
df[col]  
df.loc[label]  
df.ilo[index]
```



Analisi

```
mean()  
sum()  
groupby()
```



Pulizia

```
drop()  
fillna()  
astype()  
drop_duplicates()
```



Ora hai gli strumenti per analizzare qualsiasi dataset. **Buona fortuna, Trainer!**

